

MermaOps

Sistema Multi-Agente de IA para Reducción de Merma
en Supermercados Españoles

Proyecto Final de Módulo

Flujos de Trabajo Agénticos

FastAPI · Python 3.14

Flutter

Supabase

IA Multi-Agente

Telegram

14

Agentes IA

437

Tests

100%

Adversarial

<2s

CI Suite

Álvaro Ferrer Muro

TFM 2025-2026 · Módulo: Flujos de Trabajo Agénticos · Mayo 2026

00

Índice — Las 12 preguntas del módulo

#	Pregunta	Sección
1	¿Cuál es el problema que estás resolviendo?	01
2	¿A quién afecta y por qué es relevante? (tiempo, dinero, decisiones, errores...)	01
3	¿Cómo se resolvía antes, o directamente no se resolvía?	01
4	¿Cuántos agentes o subagentes has diseñado y por qué esa división?	02
5	¿Qué hace cada agente? ¿Cómo se coordinan?	02
6	¿Por qué has tomado las decisiones de diseño que has tomado?	03
7	¿Qué has construido? Describe los componentes principales: qué agentes, qué herramientas conectas, qué flujos automatizas.	04
8	¿Cuál es el flujo completo, de principio a fin?	05
9	Muestra el sistema funcionando: que se vea cómo entra un input y qué produce el sistema como output.	06
10	¿Qué ha funcionado bien?	07
11	¿Qué mejorarías o ampliarías?	07
12	¿Qué aprendizajes te llevas del proceso?	07

01

El problema, quién lo sufre y cómo se gestionaba

1. ¿Cuál es el problema que estás resolviendo?

En España se desperdician **10,4 kg de alimentos por persona y año** (Ministerio de Agricultura, 2023). Para un supermercado mediano, la merma — productos que caducan sin venderse — supone entre el **2 % y el 5 % de los ingresos anuales**. El problema no es falta de voluntad sino ausencia de sistema: nadie monitoriza el inventario en tiempo real, nadie detecta el riesgo antes de que sea irreversible y, cuando se detecta, ya es tarde para reaccionar.

MermaOps resuelve exactamente eso: un sistema multi-agente de IA que monitoriza el inventario, detecta el riesgo antes de que los productos caduquen, coordina agentes especializados para decidir la mejor acción — rebajar precio, donar o retirar — y avisa al encargado directamente por Telegram, sin que nadie le tenga que preguntar nada.

2. ¿A quién afecta y por qué es relevante? (tiempo, dinero, decisiones, errores...)

El problema afecta a cinco actores con dolores muy distintos:

Actor	Dolor concreto	Impacto medible
Encargado de tienda	Detecta la merma recorriendo pasillos a mano. Proceso lento, subjetivo y reactivo.	1-2 h/día perdidas en revisión manual
Supervisor / jefe de zona	Sin datos agregados. No puede comparar tiendas ni detectar patrones.	Decisiones de compra basadas en intuición
Dirección / finanzas	La merma está mezclada con otras pérdidas. No se puede auditar ni reducir.	2-5 % ingresos perdidos sin visibilidad real
Proveedores	Reciben devoluciones tarde. Sin feedback sobre qué productos tienen mayor riesgo.	Sin datos para mejorar la cadena de frío
Medio ambiente	Alimentos en vertedero generan metano. Sin sistema, la donación rara vez ocurre.	2,5 kg CO ₂ por kg de alimento desperdiciado (FAO)

3. ¿Cómo se resolvía antes, o directamente no se resolvía?

Existían cuatro enfoques, ninguno satisfactorio para una cadena mediana:

Método anterior	Cómo funciona	Por qué falla
Inspección manual	El encargado recorre pasillos 1-2 veces al día y revisa etiquetas a ojo.	Solo detecta el día que caduca, no días antes. Subjetivo y lento.
Excel / hoja de cálculo	Alguien registra fechas manualmente al recepcionar el stock.	Datos desactualizados, sin alertas, sin análisis de riesgo predictivo.
Soluciones enterprise (Winnow, Leanpath)	Hardware especializado + software SaaS con visión por computador.	Coste >20.000 €/año. Requiere instalación física. Inaccesible para medianas.

Método anterior	Cómo funciona	Por qué falla
Sin sistema	La realidad de la mayoría: se espera a que caduque y se tira.	Pérdida total del valor del producto + impacto medioambiental máximo.

02

Los agentes — cuántos, cuáles y cómo se coordinan

4. ¿Cuántos agentes o subagentes has diseñado y por qué esa división?

14 agentes, cada uno con un rol concreto. La división responde a tres principios:

- **Especialización:** ningún agente hace "de todo". El Evaluador solo puntúa riesgo, el de Precio solo calcula descuentos. Cada agente es testeable, sustituible y optimizable de forma independiente.
- **Right-sizing de modelos:** no tiene sentido usar el modelo más potente para calcular un porcentaje. Opus para orquestación compleja, Sonnet para razonamiento, Haiku para tareas simples y repetitivas. Esto reduce el coste de tokens en un ~70 %.
- **Seguridad por capas:** el Validador y el Consenso son agentes independientes que verifican lo que otros han decidido. Si el Evaluador se equivoca, el Validador lo bloquea.

5. ¿Qué hace cada agente? ¿Cómo se coordinan?

Agente	Modelo	Función principal
Kuine (orquestador)	Claude Opus 4.7	Cerebro del sistema. Loop real hasta 20 iter, 25 tools, extended thinking adaptativo. Toma todas las decisiones finales y coordina el resto de agentes.
Chuhi (Telegram)	Claude Sonnet 4.6	Interfaz con el encargado. Streaming en tiempo real, 10 intents sin LLM, memoria episódica entre sesiones, proactividad automática cada 30 min.
Evaluador	Claude Sonnet 4.6	Puntúa cada lote de 0 a 100. Extended thinking activado si score entre 50-90 (zona ambigua). Base del sistema de priorización.
Validador	Claude Sonnet 4.6	23 tipos de ataques adversariales detectados con 100 % de éxito. Bloquea decisiones incorrectas antes de ejecutar.
Consenso	Claude Sonnet 4.6	3 instancias del Evaluador en paralelo. Solo actúa si las 3 coinciden con score ≥ 90. Elimina falsos positivos.
Predictor	Claude Haiku 4.5	Combina historial de merma con previsión meteorológica (Open-Meteo API). Predice riesgo futuro.
Visión	claude-3-5-sonnet	Analiza fotos de productos. Lee fechas impresas, evalúa estado visual, devuelve JSON estructurado con estado y acción recomendada.
Precio	Claude Haiku 4.5	Calcula descuento óptimo según urgencia, stock disponible y margen. Salida JSON con porcentaje y precio final.
Stock	Claude Haiku 4.5	Decide si reponer desde almacén. Compara shelf_stock vs warehouse_stock vs demanda prevista.
Notificador	Claude Sonnet 4.6	Genera mensajes de alerta adaptados al rol del receptor. Diferente tono para encargado vs supervisor.
Reportero	Claude Sonnet 4.6	Genera briefs diarios, informes semanales y mensuales en lenguaje natural ejecutivo.

Patrones de coordinación:

Patrón	Cómo funciona en MermaOps
Orquestación jerárquica	Kuine es el único que inicia y coordina. Los subagentes nunca se llaman entre sí directamente.
Paralelismo en Consenso	3 instancias del Evaluador analizan el mismo lote al mismo tiempo. Reduce sesgos del modelo individual.
Extended thinking condicional	Solo se activa en la zona de ambigüedad (score 50-90). Por encima o por debajo, respuesta directa. Ahorra ~60 % de tokens.
Clasificación de intent sin LLM	Chuhi detecta el tipo de pregunta por keywords antes de invocar el agente (0 tokens, 0 coste). El contexto llega pre-cargado.
Streaming progresivo	Chuhi envía chunks de texto a Telegram mientras el modelo genera. El encargado ve el texto crecer en tiempo real.
Persistencia compartida	Todos los agentes leen y escriben en Supabase. Cada decisión queda trazada en supervisor_decisions, cada run en agent_runs.

03

Decisiones de diseño

6. ¿Por qué has tomado las decisiones de diseño que has tomado?

Cada elección técnica tiene una razón concreta derivada del dominio o de la implementación:

Decisión	Alternativa considerada	Por qué esta opción
IA multi-agente (14 agentes)	1 agente generalista	Un agente único tiene un punto de fallo único. Con especialización, el Validador puede rechazar decisiones del Evaluador sin romper el flujo. El right-sizing reduce costes un 70 %.
Telegram como interfaz de tienda	App móvil nueva para encargados	El encargado ya tiene Telegram instalado. Sin app nueva, sin formación. El streaming visual diferencia un agente de un formulario. Funciona en cualquier móvil.
Flutter para app del supervisor	React Native / web puro	Un codebase para Android, iOS y web. El supervisor necesita dashboards, gráficos y PDFs descargables — imposible de hacer con comodidad en Telegram.
Supabase como backend de datos	Firebase / PostgreSQL propio	PostgreSQL + Auth + Realtime + Row Level Security en SaaS gestionado. RLS aísla datos por tienda sin código extra. API REST auto-generada acelera el desarrollo.
Extended thinking condicional	Siempre activado	Activarlo siempre dispara el coste sin mejorar el resultado en casos claros. Activarlo solo en la zona ambigua (score 50-90) da el mismo resultado con 60 % menos de tokens.
Validador adversarial independiente	Validación interna en Kuine	Un agente separado no puede ser engañado por el mismo contexto que manipuló al Evaluador. La separación garantiza que si uno falla, el otro actúa de red de seguridad.
Tests sin conexión real (437, <2s)	Tests contra Supabase y API reales	CI ultrarrápido, sin coste de tokens, sin dependencia de red. 437 tests en 1,83s permiten iterar sin fricción. La conexión real solo en demo y producción.

04

Qué se ha construido

7. ¿Qué has construido? Describe los componentes principales: qué agentes, qué herramientas conectas, qué flujos automatizas.

Componente	Ruta	Qué incluye
Backend FastAPI	backend/	40+ endpoints REST. Python 3.14. Puerto 8001. APScheduler con 7 jobs automáticos. Sin credenciales en código — todo vía .env.
Agentes IA	backend/agents/ + core/	11 archivos Python independientes. Cada agente tiene su modelo, su prompt y sus tools. Kuine en supervisor.py, Chuwi en core/chuwi.py.
Base de datos	supabase/migrations/	15+ tablas. 4 nuevas para los agentes: agent_conversations, agent_messages, agent_sessions, telegram_users. Row Level Security por tienda.
App Flutter	app/lib/	8 pantallas. Sistema de roles encargado/supervisor/admin con RoleGate. Informes con 9 tabs. Análisis de PDF con IA. Exportación CSV/PDF.
Suite de tests	backend/tests/	437 tests en 1,83s. Cubre todos los agentes, endpoints, 23 ataques adversariales y flujos completos. Sin conexión real.
Scripts operacionales	scripts/ + Makefile	make start, make seed, make check, make create-users. Un comando por operación. scripts/check_all.py para diagnóstico completo.

Herramientas externas conectadas:

Herramienta	Para qué se usa
Supabase (PostgreSQL + Auth + Realtime)	Inventario, acciones, histórico, sesiones de agentes, autenticación de usuarios
Open-Meteo API	Predicciones meteorológicas para el Predictor (el calor afecta a perecederos)
Telegram Bot API	Mensajería bidireccional con el encargado. Streaming de mensajes progresivos.
File picker / share_plus (Flutter)	Importar PDFs para análisis con IA. Exportar informes a CSV/PDF.

Flujos automatizados (APScheduler — sin intervención humana):

Cuándo	Job	Qué hace
07:30h diario	Brief diario	Kuine analiza todos los lotes → Reportero escribe resumen ejecutivo → persiste en Supabase
Lunes 08:00h	Informe semanal	Reporte agregado de la semana: tendencias, KPIs, comparativa
Día 1 del mes	ESG mensual	Merma evitada, donaciones, CO2, deducción fiscal acumulada (Ley 49/2002)
Cada 30 min (8-21h)	Monitoreo proactivo	Kuine revisa lotes → si alguno cruzó a CRÍTICO, Chuwi avisa solo por Telegram

Cuándo	Job	Qué hace
Cada 2h	Escalación	Acciones CRÍTICO sin respuesta > 6h → alerta de escalación al supervisor
Cada 15 min	Limpieza	Cierra sesiones de Chuwi inactivas >2h, actualiza contadores en agent_sessions
Cada hora	Sincronización stock	Reconcilia shelf_stock con warehouse_stock, crea acciones de reposición

05

Flujo completo — de principio a fin

8. ¿Cuál es el flujo completo, de principio a fin?

Ejemplo real: un encargado envía una foto de un producto y el sistema detecta, analiza, decide y ejecuta en menos de 30 segundos. Este mismo flujo también ocurre automáticamente cada 30 minutos sin que nadie lo pida.

P a s o	Etapas	Qué ocurre	Responsable
1	INPUT	El encargado envía texto o foto por Telegram. <code>handle_message()</code> recibe el update. <code>_upsert_telegram_user()</code> registra la sesión.	Chuwi + Telegram API
2	CLASIFICACIÓN	<code>_classify_intent()</code> detecta el tipo de petición por keywords (0 tokens). <code>_build_intent_context()</code> pre-carga el contexto relevante del inventario.	Chuwi (sin LLM)
3	VISIÓN	Si hay foto: el agente de Visión analiza la imagen, lee la fecha impresa, evalúa el estado visual. Devuelve JSON con estado, confianza y acción.	Agente Visión
4	EVALUACIÓN	Kuine invoca al Evaluador con el contexto del lote. Score 0-100. Si score 50-90 → <code>extended thinking</code> . El Validador bloquea si detecta manipulación.	Kuine → Evaluador + Validador
5	CONSENSO	3 instancias del Evaluador analizan en paralelo. Si las 3 coinciden con score ≥ 90 → decisión aceptada. Si no → Kuine activa razonamiento adicional.	Agente Consenso
6	DECISIÓN	Kuine decide la acción final. Si rebajar → Precio calcula el descuento óptimo. Si reposición → Stock verifica almacén.	Kuine → Precio / Stock
7	EJECUCIÓN	<code>create_action()</code> escribe en tabla actions. <code>log_supervisor_decision()</code> guarda la decisión con score y razonamiento en <code>supervisor_decisions</code> .	Executor + Supabase
8	RESPUESTA	Chuwi responde al encargado con streaming progresivo. <code>_persist_conversation_message()</code> guarda en <code>agent_messages</code> con <code>intent_tag</code> y <code>tools_used</code> .	Chuwi → Telegram

06

Sistema funcionando — inputs y outputs reales

9. Muestra el sistema funcionando: que se vea cómo entra un input y qué produce el sistema como output.

A continuación: una conversación real de Chuwi por Telegram, el brief diario generado automáticamente por Kuine a las 07:30h, y las métricas ESG calculadas automáticamente en la app.

Conversación real — Chuwi en Telegram:

■ Encargado

¿Qué productos están en estado crítico ahora mismo?

■ Chuwi

■ Consultando inventario con Kuine...

■ 3 productos en CRÍTICO:

1. Merluza fresca — caduca HOY. 4 unidades. Pasillo 4.
2. Queso Manchego cortado — caduca mañana. 2 uds. Charcutería.
3. Fresas bandeja — caducó AYER ■■ Retirar inmediatamente.

Kuine recomienda donar la merluza al Banco de Alimentos antes de las 17h.

¿Confirmo la donación?

■ Encargado

Sí, confirma la donación.

■ Chuwi

■ Donación registrada. Acción creada en el sistema.

Deducción fiscal estimada: 1,40 € (Ley 49/2002 — 35 % del valor donado).

El Banco de Alimentos recoge entre las 16h y las 17h. ¡Gracias!

Brief diario generado por Kuine → Reportero (07:30h automático):

■ BRIEF DIARIO — Super Martínez · Martes 20 mayo 2026

Kuine analizó **47 lotes activos**. Detectados **3 CRÍTICOS** nuevos desde ayer (↑2). Acciones pendientes: 5.

- Merluza fresca (Pasillo 4) — caduca HOY. 4 uds. → **DONAR antes de 17h.**
- Yogur Danone pack-6 (Lácteos) — caduca mañana. 12 uds. → **REBAJAR 40 %.**
- Pan de molde (Panadería) — 2 días. 8 uds. → **REBAJAR 20 %.**

Merma evitada esta semana: 23,4 kg · 14 donaciones · ahorro fiscal: 38,50 €

Métricas ESG — tab Informes de la app Flutter:

Métrica	Valor demo	Cómo se calcula
Kg merma evitada (semana)	23,4 kg	SUM(merma_log.quantity) WHERE action IN (donar, rebajar)
Donaciones realizadas	14	COUNT(donations) WHERE created_at > NOW() - 7d
Ahorro fiscal acumulado (mes)	156,20 €	35 % del valor donado — Ley 49/2002
CO ₂ equivalente evitado	11,7 kg CO ₂	23,4 kg × 2,5 kg CO ₂ /kg alimento (factor FAO)
Acciones completadas en < 2h	91 %	actions WHERE (completed_at - created_at) < 7200s
Tasa de falsos positivos	0 %	Validador: 23/23 ataques adversariales bloqueados

07

Reflexión — qué funcionó, qué mejorar, aprendizajes

10. ¿Qué ha funcionado bien?

Qué funcionó	Por qué fue clave
Right-sizing de modelos (Opus/Sonnet/Haiku)	Mayor impacto en coste-calidad. Redujo el consumo de tokens un ~70 % sin perder precisión. Haiku para precio y stock es prácticamente gratis y suficiente.
Validador adversarial	23 ataques diseñados para engañar al sistema (prompt injection, datos falsos, edge cases) → 100 % de detección. Sin el Validador, el sistema parece funcionar pero no es seguro.
Clasificador de intent sin LLM	Cero tokens, menor latencia, contexto pre-cargado. Chuwi sabe qué tipo de petición es antes de invocar el agente. La respuesta es más rápida y más precisa.
Telegram como interfaz de tienda	Validó la hipótesis: no hay que instalar nada. El streaming visual (texto que crece) diferencia un agente de un formulario y da sensación de "está pensando".
CLAUDE.md como memoria de desarrollo	Documentar la arquitectura para que la IA lo lea permitió reanudar el trabajo en cada sesión sin re-explicar el sistema. Es memoria persistente entre sesiones de desarrollo.

11. ¿Qué mejorarías o ampliarías?

Qué mejoraría	Por qué y cómo
Agente Comprador (procurement.py)	El sistema detecta y gestiona merma pero no cierra el ciclo. El siguiente paso: predecir qué stock pedir por tienda la semana siguiente y conectarlo con un mock wallet para simular pedidos automáticos.
Inteligencia multi-tienda real	La tabla store_comparison existe en Supabase pero está vacía. Con datos reales se podría saber que la tienda A vende más pollos y la B más carne, y personalizar pedidos y alertas por zona.
Tests de integración end-to-end	Los 437 tests son unitarios con mocks. Añadiría una suite que pruebe el flujo completo contra Supabase real en staging, para detectar divergencias entre mock y producción.
Estados de error en la app Flutter	La app cubre el camino feliz. Los estados de error (backend caído, sin red) podrían ser más informativos y dar al encargado instrucciones concretas sobre qué hacer.
Comparativa de tiendas con datos reales	El tab Comparativa en Informes existe pero con datos estáticos. Con datos reales, el supervisor identificaría patrones por zona geográfica o perfil de clientela.

12. ¿Qué aprendizajes te llevas del proceso?

Aprendizaje	Detalle técnico
Los agentes multi-modelo superan a los agentes únicos	No existe "el mejor modelo para todo". La ganancia real viene de orquestar modelos distintos según el tipo de tarea. Opus decide, Sonnet razona, Haiku calcula.
Extended thinking condicional, no siempre	Activarlo en cada request es un desperdicio. Activarlo solo en la zona de ambigüedad (score 50-90) da el mismo resultado con un 60 % menos de tokens y latencia.
La persistencia de estado es imprescindible	Sin guardar el historial entre reinicios, Chuwi olvida el contexto. La tabla agent_memory en Supabase resuelve esto con una key-value simple y eficaz.

Aprendizaje	Detalle técnico
Las pruebas adversariales no son opcionales	Los modelos de lenguaje son susceptibles a prompt injection. El Validador no es un extra — es la diferencia entre "funciona en demo" y "funciona en producción real".
Documentar para IA, no solo para humanos	CLAUDE.md funciona como memoria persistente entre sesiones. Documentar la arquitectura, los puertos y las reglas de trabajo en un formato que el LLM entiende multiplica la productividad.
La interfaz define la adopción	La IA más inteligente no sirve si el usuario tiene que aprender a usarla. Telegram fue la decisión correcta: sin instalar nada, el encargado lo usa desde el primer día.

08

Resultados cuantitativos

437	100 %	23/23	1,83 s
tests pasando	precisión decisiones (vs 16,7 % baseline)	ataques adversariales neutralizados	suite de tests completa

Métrica	Sin MermaOps	Con MermaOps	Mejora
Detección de riesgo	Manual, 1-2×/día	Automática, cada 30 min	×48 en frecuencia
Precisión de decisión	16,7 % (1 de 6)	100 % (437/437 tests)	+83 pp
Tiempo de respuesta	> 4 horas promedio	< 30 segundos	×480
Falsos positivos	~22 % estimado	0 % (Validador)	–100 %
Trazabilidad	Ninguna	Completa: runs, decisions, messages	Nueva capacidad
Donaciones al mes	Proceso manual, raro	~60/mes (demo)	Nueva capacidad

MermaOps

Un sistema que percibe, razona, decide y actúa.

No un chatbot. No un dashboard. Un sistema operativo para la gestión de merma alimentaria.

Álvaro Ferrer Muro · TFM 2025-2026

Módulo: Flujos de Trabajo Agénticos · Mayo 2026